

Customer Order

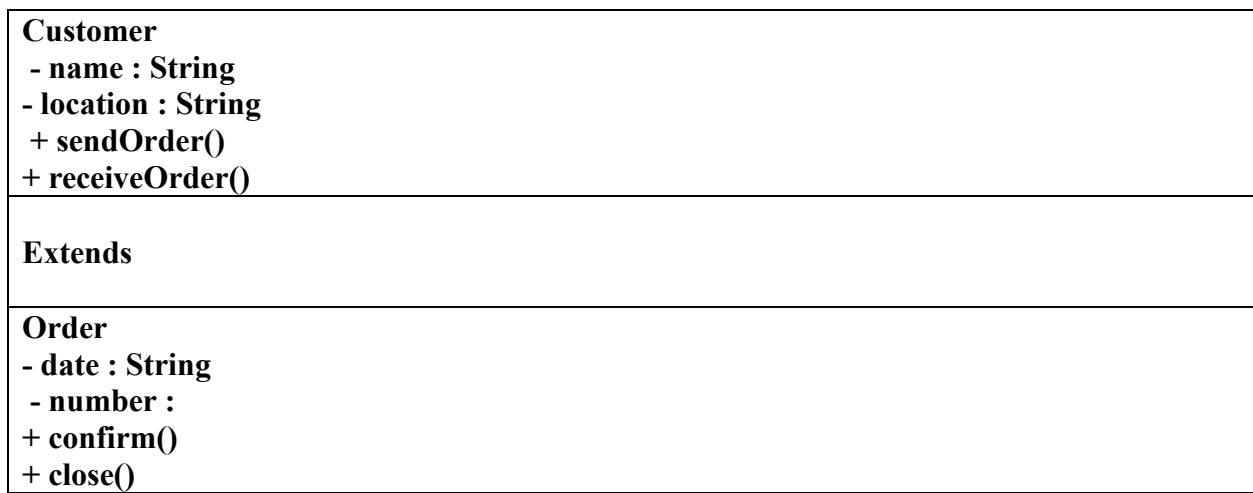
1. Objective

To develop a Java program that demonstrates **multilevel inheritance** using the classes Customer, Order, SpecialOrder, and NormalOrder.

2. Algorithm

1. Start the program.
2. Create a Customer class with name and location as data members.
3. Define methods sendOrder() and receiveOrder() in the Customer class.
4. Create an Order class that extends Customer.
5. Add date and number data members to the Order class.
6. Define confirm() and close() methods in the Order class.
7. Create a SpecialOrder class that extends Order and define the dispatch() method.
8. Create a NormalOrder class that extends Order and define dispatch() and receive() methods.
9. Create objects of SpecialOrder and NormalOrder in the main() method.
10. Call the inherited and class-specific methods.
11. Display the order details.
12. Stop the program.

3. UML Class Diagram



Extends

SpecialOrder + dispatch()	NormalOrder + dispatch() + receive()
--	---

CustomerOrder -> creates objects -> Special/Normal orders

4. Source Code

// Customer class

```
class Customer{
```

```
    // Data members
```

```
    String name;
```

```
    String location;
```

```
    // Constructor of Customer
```

```
    Customer(String name, String location){
```

```
        this.name = name;
```

```
        this.location = location;
```

```
    }
```

```
    // Method to send an order
```

```
    void sendOrder(){
```

```
        System.out.println(name + " sends the order.");
```

```
    }
```

```
    // Method to receive an order
```

```
    void receiveOrder(){
```

```
        System.out.println(name + " receives the order.");
```

```
    }
```

```
}
```

```
// Order class inherits Customer
class Order extends Customer{
    // Data members of Order
    String date;
    int number;
    // Constructor of Order
    Order(String name, String location, String date, int number){
        super(name, location);
        this.date = date;
        this.number = number;
    }
    // Method to confirm order
    void confirm(){
        System.out.println("Order " + number + " is confirmed.");
    }
    // Method to close order
    void close(){
        System.out.println("Order " + number + " is closed.");
    }
}

// SpecialOrder inherits Order
class SpecialOrder extends Order{
    // Constructor of SpecialOrder
    SpecialOrder(String name, String location, String date, int number){
        super(name, location, date, number);
    }
    // Method to dispatch special order
```

```

    void dispatch(){
        System.out.println("Special order " + number + " is dispatched.");
    }
}

// NormalOrder inherits Order
class NormalOrder extends Order{
    // Constructor of NormalOrder
    NormalOrder(String name, String location, String date, int number){
        super(name, location, date, number);
    }
    // Method to dispatch normal order
    void dispatch(){
        System.out.println("Normal order " + number + " is dispatched.");
    }
    // Method to receive normal order
    void receive(){
        System.out.println("Normal order " + number + " is received.");
    }
}

// Main class
class CustomerOrder{
    public static void main(String args[]){
        // Creating SpecialOrder object
        SpecialOrder s = new SpecialOrder("Mohana", "Hyderabad", "26-08-2026", 101);
        System.out.println("----- SPECIAL ORDER -----");
        // Calling methods inherited from Customer
        s.sendOrder();
    }
}

```

```

// Calling methods inherited from Order
s.confirm();
s.close();
// Calling method of SpecialOrder
s.dispatch();
System.out.println();
// Creating NormalOrder object
NormalOrder n = new NormalOrder(
    "Rahul", "Warangal", "26-08-2026", 102);
System.out.println("----- NORMAL ORDER -----");
// Calling methods inherited from Customer
n.sendOrder();
// Calling methods inherited from Order
n.confirm();
n.close();
// Calling methods of NormalOrder
n.dispatch();
n.receive();
}
}

```

5. Output

----- SPECIAL ORDER -----

Mohana sends the order.

Order 101 is confirmed.

Order 101 is closed.

Special order 101 is dispatched.

----- NORMAL ORDER -----

Rahul sends the order.

Order 102 is confirmed.

Order 102 is closed.

Normal order 102 is dispatched.

Normal order 102 is received.